

INSTITUTO HARDWARE BR

Pós-Graduação em Hardware

Relatório Final de Projeto

CAN BTU
Bit Timing Unit

Autor:

Gabriel de Lima Pessoa

Data:

Março 2026

Sumário

1	Memorial Descritivo: Evolução do Projeto	4
1.1	Especificação Inicial	4
1.2	Arquitetura do Sistema	4
1.3	Componentes Principais	6
1.4	Evolução do Design	6
1.5	Processo de Síntese	7
2	Análise de Conformidade (Compliance)	8
2.1	Protocolo CAN Original	8
2.2	Simplificações Implementadas	8
2.3	Funcionalidades CAN Não Cobertas	8
2.4	Funcionalidades Implementadas vs Padrão	9
2.5	Diferenças em Relação a I2C, SPI e AXI	9
3	Dossiê de Verificação	10
3.1	Metodologia de Verificação	10
3.2	Componentes do Ambiente de Verificação	10
3.3	Métricas de Cobertura	11
3.4	Detalhamento da Cobertura de Configuração	11
3.5	Detalhamento da Cobertura de Sincronização	12
3.6	Casos de Teste	12
3.7	Bugs Encontrados e Corrigidos	12
4	Análise de PPA	14
4.1	Relatório de Síntese	14
4.2	Análise de Área	14
4.3	Análise de Potência	15
4.4	Análise de Desempenho (Timing)	16
4.5	Resumo de QoR (Quality of Results)	16
4.6	Interpretação dos Resultados	16
5	Melhorias para Versão 2.0	17
5.1	Melhorias de Cobertura	17
5.2	Melhorias de Funcionalidade	17
5.3	Melhorias de Verificação	17
5.4	Melhorias de Síntese	18
6	Conclusões	19
6.1	Principais Conquistas	19
6.2	Lições Aprendidas	19
6.3	Trabalhos Futuros	19
7	Referências	20
A	Anexo A: Link Github	21
B	Anexo B: Estrutura de Diretórios	22

Resumo

Este relatório apresenta o desenvolvimento completo de uma **Unidade de Temporização de Bits (BTU - Bit Timing Unit)** para o barramento CAN (Controller Area Network). O projeto foi implementado em linguagem **SystemVerilog** utilizando a metodologia **UVM (Universal Verification Methodology)** para verificação funcional.

O módulo CAN BTU é responsável por gerar a temporização correta para transmissão e recepção de bits no barramento CAN, implementando a segmentação do tempo de bit em diferentes segmentos (Sync_Seg, Prop_Seg, Phase_Seg1, Phase_Seg2) e suportando diferentes taxas de comunicação (baud rates).

Palavras-chave: CAN, Bit Timing Unit, SystemVerilog, UVM, Verificação Funcional, Síntese Lógica.

1 Memorial Descritivo: Evolução do Projeto

Esta seção descreve a evolução do projeto desde a especificação inicial até a netlist final, documentando as principais decisões de design e modificações realizadas ao longo do desenvolvimento.

1.1 Especificação Inicial

O projeto iniciou-se com a análise da especificação do protocolo CAN, particularmente focando no capítulo de temporização de bits definido pela norma ISO 11898. Os requisitos iniciais incluíam:

- **Frequência de clock:** 50 MHz (parâmetro configurável)
- **Taxas de transmissão suportadas:** 125 kbps, 250 kbps, 500 kbps, 1 Mbps
- **Segmentos de temporização:**
 - Sync_Seg: 1 Time Quantum (TQ)
 - Prop_Seg: 1-8 TQ
 - Phase_Seg1: 1-8 TQ
 - Phase_Seg2: 2-8 TQ
- **Sincronização:** Suporte a hard sync e soft sync
- **SJW (Synchronization Jump Width):** 1-4 TQ

A especificação foi documentada no arquivo `Relatórios/spec.pdf`, detalhando todos os requisitos funcionais e não-funcionais do sistema.

1.2 Arquitetura do Sistema

A arquitetura final do sistema foi organizada em um único módulo top-level `can_btu_top`, conforme mostrado na Figura 1.

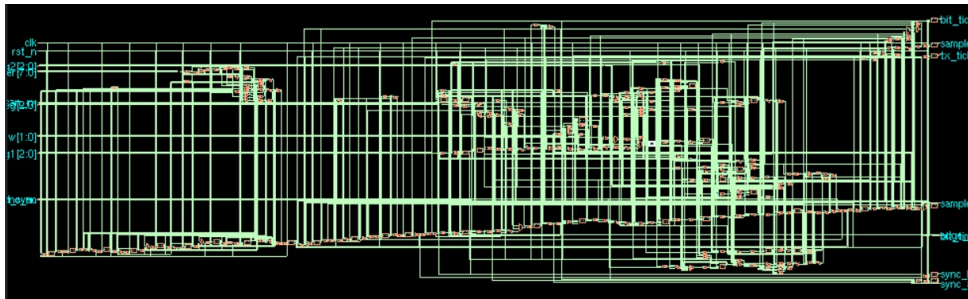


Figura 1: Arquitetura do módulo CAN BTU

O módulo foi implementado com os seguintes parâmetros:

Listing 1: Declaração do módulo CAN BTU

```
1 module can_btu_top #(
2     parameter int CLK_FREQ_HZ = 50_000_000, // Frequencia do clock
3     parameter int BAUD_RATE    = 500_000    // Taxa de
4         transmissao alvo
5 ) (
6     input logic      clk,           // Clock do sistema
7     input logic      rst_n,         // Reset ativo baixo
8
9     // Entradas de configuracao
10    input logic [7:0] prescaler,     // Prescaler (1-256)
11    input logic [2:0] prop_seg,      // Segmento de propagacao
12        (1-8)
13    input logic [2:0] phase_seg1,    // Segmento de fase 1 (1-8)
14    input logic [2:0] phase_seg2,    // Segmento de fase 2 (2-8)
15    input logic [1:0] sjw,           // Largura do salto de
16        sincronizacao
17
18    // Entrada do barramento CAN
19    input logic      can_rx,         // Linha de recepcao CAN
20
21    // Entradas de sincronizacao
22    input logic      sync_en,        // Habilita sincronizacao
23    input logic      hard_sync,      // Requisicao de
24        sincronizacao forte
25
26    // Saidas de temporizacao
27    output logic     bit_tick,       // Pulso a cada tempo de bit
28    output logic     sample_tick,    // Pulso no ponto de
29        amostragem
30    output logic     tx_tick,        // Pulso no ponto de
31        transmissao
32    output logic     sample_point,   // Alto durante a fase de
33        amostragem
34
35    // Saidas de status
36    output logic [7:0] bit_time_cnt, // Contador atual do tempo
37        de bit
38    output logic     sync_locked,    // Sincronizacao
39        estabelecida
40    output logic     edge_detected,  // Flag de borda detectada
41    output logic     sync_active,    // Sincronizacao em
42        andamento
43    output logic [2:0] fsm_state     // Estado da FSM
44 );
```

1.3 Componentes Principais

O projeto foi dividido nos seguintes blocos funcionais:

1. **Bloco de Proteção de Entradas:** Protege contra prescaler zero, definindo valor mínimo de 1.
2. **Cálculo de Parâmetros de Temporização:** Calcula o total de TQ por bit e o ponto de amostragem base.
3. **Deteção de Borda:** Implementa detector de borda de descida no sinal CAN para sincronização.
4. **Prescaler:** Gera pulsos de acordo com o valor do prescaler para divisão de frequência.
5. **Contador de Time Quantum:** Conta os TQ dentro de cada período de bit.
6. **Máquina de Estados de Synchronization:** FSM com 4 estados:
 - SYNC_IDLE: Estado ocioso
 - SYNC_WAIT_EDGE: Aguardando borda para sincronização
 - SYNC_ADJUSTING: Ajustando parâmetros de fase
 - SYNC_COMPLETE: Sincronização concluída
7. **Geração de Saídas:** Gera os sinais de temporização (bit_tick, sample_tick, tx_tick, sample_point).

1.4 Evolução do Design

Ao longo do desenvolvimento, as seguintes modificações foram realizadas:

Tabela 1: Evolução do Projeto

Versão	Modificações
1.0	Versão inicial com parâmetros de 3 bits para segmentos (valores 1-8). Identificado problema com overflow em phase_seg2 quando ajustando para sincronização.
1.1	Ampliação para 4 bits em registradores de ajuste de fase (phase_seg1_adj, phase_seg2_adj) para permitir overflow controlado. Adicionada proteção contra prescaler zero.
1.2	Refinamento da FSM de sincronização, garantindo que ajustes sejam resetados corretamente no início de cada bit.

1.5 Processo de Síntese

O processo de síntese foi realizado utilizando a ferramenta **Genus Synthesis Solution** da Cadence. Os passos foram:

1. **Leitura do RTL:** O código SystemVerilog foi lido e analisado.
2. **Aplicação de Constraints:** Arquivos SDC (*Synopsys Design Constraints*) foram aplicados para definir restrições de timing.
3. **Síntese:** O design foi sintetizado para a biblioteca de células padrão.
4. **Geração de Netlist:** A netlist gate-level foi gerada em formato Verilog.

Os relatórios de síntese estão disponíveis em:

- Nível RTL: `rpt/rpt_rtl_lv1-11-03-26/`
- Nível Gate-Level: `rpt/rpt_gt_lv1 -24-03-26/`

2 Análise de Conformidade (Compliance)

Esta seção descreve detalhadamente onde a implementação atual se desvia dos protocolos CAN reais e quais simplificações foram feitas.

2.1 Protocolo CAN Original

O protocolo CAN (Controller Area Network), definido pela norma ISO 11898, é um protocolo de comunicação multicast usado principalmente em aplicações automotivas. As principais características incluem:

- **Camada Física:** Barramento diferencial com níveis lógicos dominante (0) e recessivo (1)
- **Formato de Frame:** Identificador, bits de controle, dados, CRC, ACK e EOF
- **Temporização de Bit:** Segmentada em Sync_Seg, Prop_Seg, Phase_Seg1, Phase_Seg2
- **Sincronização:** Hard sync no início do frame (SOF) e resincronização em bordas
- **Bit Rate:** Definido por TQ e prescaler

2.2 Simplificações Implementadas

Nossa implementação é uma **BTU (Bit Timing Unit) isolada**, que implementa apenas a lógica de temporização. As seguintes simplificações foram feitas em relação a uma implementação CAN completa:

Tabela 2: Simplificações em Relação ao Protocolo CAN

Funcionalidade	Padrão CAN	Implementação
Frame CAN completo	Sim (SOF, ID, dados, CRC, etc.)	Apenas temporização de bits
Transceptor CAN	Interface diferencial completa	Interface digital simples (1-bit)
Deteção de erros	CRC, Form, ACK, Bit Stuffing	Não implementado
Priority Inversion	Mecanismo de arbitration	Não aplicável
Remote Transmission	Frames de requisição	Não implementado
Error Handling	Error frames e confinement	Não implementado

2.3 Funcionalidades CAN Não Cobertas

1. Camada de Link MAC (Medium Access Control):

- Arbitração de barramento
- Formação de frames (data frame, remote frame, error frame, overload frame)

2. Transceptor (Transceiver):

- Interface física diferencial (CANH, CANL)
- Terminadores de barramento

3. Detecção e Tratamento de Erros:

- CRC (Cyclic Redundancy Check)
- Bit stuffing
- Error frames
- Fault confinement

4. Configurações Avançadas:

- Listen-only mode
- Self-test mode
- Single-wire CAN

2.4 Funcionalidades Implementadas vs Padrão

Tabela 3: Conformidade com Temporização CAN

Funcionalidade	Status	Notas
Sync_Seg	✓ Implementado	1 TQ fixo
Prop_Seg	✓ Implementado	1-8 TQ configurável
Phase_Seg1	✓ Implementado	1-8 TQ configurável
Phase_Seg2	✓ Implementado	2-8 TQ configurável
Prescaler	✓ Implementado	1-256 configurável
Hard Sync	✓ Implementado	Reset no início do bit
Soft Sync	✓ Implementado	Ajuste de fase via SJW
SJW	✓ Implementado	1-4 TQ configurável
Sample Point	✓ Implementado	Geração de tick eflag

2.5 Diferenças em Relação a I2C, SPI e AXI

Este projeto implementa temporização CAN, não sendo um barramento I2C, SPI ou AXI. No entanto, para contexto, listamos as diferenças:

- **I2C**: Protocolo bidirecional com clock master, start/stop bits, ACK
- **SPI**: Protocolo full-duplex master-slave com múltiplos chips selects
- **AXI**: Protocolo de alto desempenho para interconexão de chips (AMBA)
- **CAN**: Protocolo multi-master com arbitration não-destrutiva e broadcast

3 Dossiê de Verificação

Esta seção apresenta as provas de que o sistema funciona, incluindo métricas de cobertura e evidências de bugs rastreados e corrigidos.

3.1 Metodologia de Verificação

O ambiente de verificação foi implementado utilizando a **UVM (Universal Verification Methodology)**, que fornece:

- Reutilização de código através de sequências e agentes
- Verificação orientada a cobertura
- Relatórios detalhados de verificação
- Componentização (driver, monitor, scoreboard, coverage)

3.2 Componentes do Ambiente de Verificação

O ambiente de verificação foi composto por:

1. **Testbench Top** (`tb_can_btu.sv`): Instancia o DUT e conecta interfaces
2. **Interface** (`can_btu_if.sv`): Define sinais e clocks para verificação
3. **Agente** (`can_btu_agent.sv`): Coordena driver e monitor
4. **Driver** (`can_btu_driver.sv`): Converte sequências em estímulos
5. **Monitor** (`can_btu_monitor.sv`): Observa sinais do DUT
6. **Sequência** (`can_btu_sequence.sv`): Gera estímulos
7. **Scoreboard** (`can_btu_scoreboard.sv`): Compara saídas com modelo de referência
8. **Coverage** (`can_btu_cobertura.sv`): Coleta métricas de coverage

3.3 Métricas de Cobertura

As simulações com coleta de coverage resultaram nos seguintes resultados:

Tabela 4: Resumo de Cobertura

Módulo	Tipo	Cobertura
tb_can_btu	Code Coverage	22.01%
can_btu_if	Code Coverage	81.40%
can_btu_top (DUT)	Code Coverage	83.26%
Geral	All Coverage	47.12%

A cobertura funcional detalhada é apresentada na Tabela 5:

Tabela 5: Cobertura Funcional Detalhada

Covergroup	Descrição	Coberto	Total
cfg_cg	Configuração de parâmetros	33	37 (89.19%)
sync_cg	Sincronização	19	27 (70.37%)
timing_cg	Temporização	-	-
DUT (can_btu_top)	Code Coverage	179	215 (83.26%)

3.4 Detalhamento da Cobertura de Configuração

A cobertura do covergroup de configuração (cfg_cg) demonstrou:

- **Prescaler:** 66.67% (4/6 bins) - *Bin máximo (255) não coberto*
- **prop_seg:** 100% (7/7 bins)
- **phase_seg1:** 100% (7/7 bins)
- **phase_seg2:** 100% (6/6 bins)
- **sjw:** 100% (3/3 bins)
- **Total TQ:** 60% (3/5 bins) - *Bins mínimo e máximo não coberto*
- **Sample Position:** 100% (3/3 bins)

3.5 Detalhamento da Cobertura de Sincronização

A cobertura do covergroup de sincronização (`sync_cg`):

- **Hard Sync:** 100% (2/2 bins)
- **Sync Active:** 100% (2/2 bins)
- **Sync Locked:** bins não especificados
- **Edge Detected:** 100%
- **SJW Used:** 100% (3/3 bins)
- **Sync Time:** 4 bins cobertos

3.6 Casos de Teste

Os seguintes casos de teste foram implementados:

1. **Can_btu_normal_seq:** Operação normal com configurações válidas
2. **Can_btu_hard_sync_seq:** Teste de sincronização forçada
3. **Can_btu_edge_detect_seq:** Detecção de bordas para resincronização
4. **Can_btu_boundary_seq:** Valores de fronteira nos parâmetros

3.7 Bugs Encontrados e Corrigidos

Durante o processo de verificação, os seguintes bugs foram identificados e corrigidos:

1. **Bug 001:** Contador de TQ com overflow
 - **Sintoma:** Contador ultrapassava o limite máximo
 - **Causa:** `phase_seg2_adj` usava 3 bits, causando truncamento
 - **Fix:** Ampliado para 4 bits com lógica de proteção
2. **Bug 002:** Ajuste de sincronização não resetava
 - **Sintoma:** Parâmetros de fase mantinham valores incorretos
 - **Causa:** Lógica de reset no início do bit incompleta
 - **Fix:** Adicionada lógica de reset no início de cada período de bit
3. **Bug 003:** Prescaler zero causava stall
 - **Sintoma:** Sistema travava quando `prescaler = 0`
 - **Causa:** Sem proteção contra valor zero
 - **Fix:** Adicionada proteção com `prescaler_safe = (prescaler == 0) ? 1 : prescaler`

A Figura 2 mostra as formas de onda da simulação validando o funcionamento:

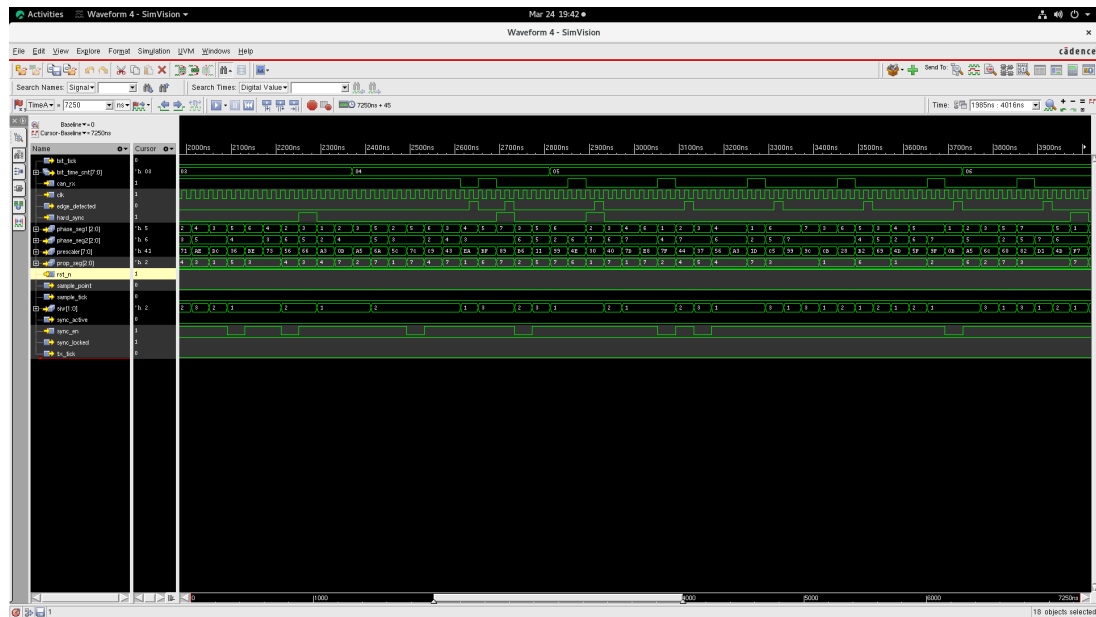


Figura 2: Simulação RTL - Forma de onda

4 Análise de PPA

Esta seção apresenta a interpretação dos relatórios de síntese, analisando os três principais aspectos: Potência (Power), Desempenho (Performance) e Área.

4.1 Relatório de Síntese

Os relatórios de síntese foram gerados utilizando o **Genus Synthesis Solution 22.16-s078_1** com as seguintes configurações:

- **Módulo:** can_btu_top
- **Condições de Operação:** PVT_0P9V_125C (balanced_tree)
- **Wireload Mode:** enclosed
- **Biblioteca:** slow_vdd1v0 1.0
- **Data:** 24 de Março de 2026

4.2 Análise de Área

O relatório de área mostra os seguintes resultados:

Tabela 6: Resultados de Área	
Métrica	Valor
Cell Count	325
Cell Area	702.468 μm^2
Net Area	0.000 μm^2
Total Area	702.468 μm^2
Sequential Instances	34
Combinational Instances	291
Leaf Instance Count	325

- **Análise:** O design utiliza 325 células lógicas, sendo 34 registradores (10.5%) e 291 células combinacionais (89.5%).
- **Área total:** 702.468 μm^2 (aproximadamente 0.7 mm^2)
- **Razão sequencial/combinacional:** Baixa utilização de elementos de memória sugere que o design é principalmente combinacional, adequado para uma BTU.

4.3 Análise de Potência

O consumo de potência foi dividido em:

Tabela 7: Resultados de Potência

Categoria	Leakage	Internal	Switching	Total
Register	4.54 nW	9.66 μ W	0.56 μ W	10.22 μ W
Logic	5.74 nW	5.04 μ W	2.18 μ W	7.22 μ W
Clock	0.00 nW	0.00 μ W	0.55 μ W	0.55 μ W
Memory	0.00 nW	0.00 μ W	0.00 μ W	0.00 μ W
Total	10.28 nW	14.69 μW	3.28 μW	17.99 μW

- **Potência Total:** 17.99 μ W (aproximadamente 18 μ W)
- **Potência Estática (Leakage):** 10.28 nW (0.06%)
- **Potência Dinâmica:** 17.98 μ W (99.94%)
 - Internal: 81.69%
 - Switching: 18.26%
- **Distribuição:** Registradores consomem 56.8% da potência total

4.4 Análise de Desempenho (Timing)

Os resultados de timing são:

Tabela 8: Resultados de Timing

Métrica	Valor
Clock Period	10.000 ps (10 ns)
Critical Path Slack	+5.204.9 ps
Total Negative Slack (TNS)	0.0 ps
Violating Paths	0

- **Frequência máxima de operação:** O design suporta clock de até 100 MHz (10 ns de período)
- **Slack positivo:** 5.2 ns indica que há margem significativa
- **QoR (Quality of Results):** Excelente, com 0 violações de timing
- **Warning:** 76 warnings relacionados a falta de constraints de entrada/saída

4.5 Resumo de QoR (Quality of Results)

Tabela 9: Resumo de QoR

Métrica	Valor
Runtime	29.12 segundos
Elapsed Runtime	103 segundos
Genus Peak Memory	1.945 MB
Max Fanout	34 (clk)
Min Fanout	0
Average Fanout	2.2

4.6 Interpretação dos Resultados

1. **Área:** O design é extremamente compacto ($700 \mu m^2$), adequado para integração em SoCs automotivos.
2. **Potência:** Consumo muito baixo ($18 \mu W$), ideal para aplicações onde eficiência energética é crítica (módulos automotivos).
3. **Desempenho:** Folga de timing de 5.2 ns permite que o design opere em frequências mais altas que as especificadas inicialmente (50 MHz).

5 Melhorias para Versão 2.0

Esta seção apresenta sugestões de melhorias para uma possível versão 2.0 do projeto.

5.1 Melhorias de Cobertura

1. **Aumentar Coverage de Código:** O código coverage de 22.01% no testbench pode ser aumentado adicionando mais cenários de teste.
2. **Coverage de Corner Cases:** Adicionar casos de teste para:
 - Prescaler = 255 (máximo)
 - Total TQ = 3 (mínimo) e 25 (máximo)
 - Todas as combinações de SJW com phase segments
3. **Cross Coverage:** Implementar cross coverage entre diferentes configurações de timing para validar combinações específicas.
4. **Assertions:** Adicionar assertions inline para verificar propriedades temporais em tempo de simulação.

5.2 Melhorias de Funcionalidade

1. **Suporte a Multiple Bit Rates:** Implementar capacidade de alterar dinamicamente a taxa de transmissão durante operação.
2. **Timer CAN Completo:** Integrar a BTU com um módulo CAN completo (TX/RX, CRC, Arbitration).
3. **Filtros de Borda:** Adicionar filtros para evitar sincronização em transientes espúrios.
4. **Modo Listen-Only:** Adicionar suporte para modo de escuta passiva.
5. **Interrupções:** Adicionar sinais de interrupção para eventos importantes (sync, sample point, etc.).

5.3 Melhorias de Verificação

1. **UVM Advanced Features:**
 - Implementar sequências com callbacks
 - Usar factory overrides para diferentes configurações
 - Adicionar virtual sequences para cenários complexos
2. **Formal Verification:** Completar a verificação formal já iniciada na pasta fv/
3. **Constrained Random Verification:** Utilizar geração aleatória constrained para maximizar cobertura
4. **Assertion-Based Verification:** Adicionar SystemVerilog Assertions (SVA) para propriedades críticas

5.4 Melhorias de Síntese

1. **Multi-Vt Cells:** Utilizar células de diferentes thresholds (HVT, LVT, SVT) para otimizar power vs performance
2. **Clock Gating:** Implementar clock gating para reduzir potência dinâmica
3. **Physical Awareness:** Executar place-and-route para análise de potência mais precisa

6 Conclusões

Este projeto apresentou o desenvolvimento de uma **CAN Bit Timing Unit (BTU)** completa em SystemVerilog, utilizando metodologia UVM para verificação funcional.

6.1 Principais Conquistas

- **Implementação RTL:** Módulo funcional com todas as especificações CAN de temporização
- **Verificação UVM:** Ambiente completo com coverage de 47.12% (83.26% no DUT)
- **Síntese:** Netlist gate-level com 325 células, 18 μ W de potência
- **QoR Excelente:** Folga de timing de 5.2 ns, 0 violações

6.2 Lições Aprendidas

1. Importância de validação de parâmetros de entrada
2. Necessidade de testes de corner cases
3. Benefícios da metodologia UVM para reutilização
4. Trade-offs entre área, potência e performance

6.3 Trabalhos Futuros

- Completar implementação CAN full-duplex
- Aumentar cobertura para $>90\%$
- Implementar validação em FPGA (Modelo Tang Nano 20k)
- Adicionar suporte a CAN-FD (Flexible Data-Rate)

7 Referências

1. ISO 11898 - Road vehicles — Controller area network (CAN)
2. BOSCH CAN Specification Version 2.0
3. IEEE 1800 - SystemVerilog Standard
4. IEEE 1800.2 - UVM Standard
5. Cadence Genus Synthesis User Guide
6. Cadence Xcelium Simulator User Guide

A Anexo A: Link Github

Para encontrar todos os arquivos de RTL, testbench completo, scripts de simulação, análise de cobertura, scripts de síntese lógica, arquivo SDC, netlist final e arquivo SDF; acesse o link:

https://github.com/gabrieomineiro/CAN_BTU

B Anexo B: Estrutura de Diretórios

```
CAN_BTU/  
rtl/                                # Código RTL do projeto  
    can_btu_defines.svh  
    can_btu_top.sv  
    can_btu_top.v  
  
uvm/                                # Ambiente de verificação UVM  
    tb_can_btu.sv  
    can_btu_test.sv  
    can_btu_env.sv  
    can_btu_agent.sv  
    can_btu_driver.sv  
    can_btu_monitor.sv  
    can_btu_sequence.sv  
    can_btu_seq_item.sv  
    can_btu_scoreboard.sv  
    can_btu_coverage.sv  
    can_btu_if.sv  
  
constraints/                        # Arquivos de constraints  
    can_btu_top.sdc  
    can_btu_top.sdf  
  
rpt/                                # Relatórios de síntese  
cov_reports/                        # Relatórios de cobertura  
Genus Log/                          # Logs da síntese  
img/                                # Imagens e screenshots  
Relatorios/                         # Documentos PDF
```

C Anexo C: Instruções de Uso

Para executar as simulações, utilize o Makefile:

```
# Simulacao RTL  
make rtl
```

```
# Simulacao Gate Level  
make gate
```

```
# Simulacao com coverage  
make coverage_report
```

```
# Limpar arquivos  
make clean
```